Operating Systems - Assignment 1

Character Device Driver Implementation

Name: Elias Joby
Roll No: B240175CS **Date:** 01-April-2026

---

1. ***Problem Statement***

The objective of this assignment is to implement a Linux Character Device Driver with the following features:

- Accept two parameters:
  - `kernel_version` → specifies kernel version
  - `time` → time limit in seconds
- The module should only load if the given kernel version matches the compilation kernel version.
- On successful insertion:
  - Print major number, minor number, and timer value using `dmesg`.
- Perform two actions in order:
  - Read from device
  - Write username to device
- On module removal:
  - Verify if actions were completed in order and within time
  - Print success or failure message accordingly

2. ***Methodology***

The implementation was carried out using the following steps:

- **Kernel Module Development**
- A loadable kernel module (`mymodule.c`) was written.
- Parameters were passed using:

```
/* Module parameters */
module_param(kernel_version_0, int, 0000);
module_param(kernel_version_1, int, 0000);
module_param(time, int, 0000);
```

- **Version Validation**

- Kernel version was extracted using: `LINUX_VERSION_CODE`
- Compared with user input (`kernel_version_0`, `kernel_version_1`)

- **Character Device Creation**

- Device registered using: `register_chrdev(`
- Major number dynamically allocated

- **Device Operations**

- *Read Operation*
  - Sets `read_done = 1`
- *Write Operation*
  - Allowed only after read
  - Stores username using `copy_from_user()`

- **Time Tracking**

- Start time recorded using: `jiffies`
- Elapsed time computed during module removal

- **Validation Logic**

- On `rmmod`, checks:
  - read → write order
  - within time limit
- Prints:
  - Success + username OR
  - Failure message

---

3. ***Explanation of Working***

The driver follows this execution flow:

insmod → version check → register device → start timer→ read → write → rmmod → validation

- The implemented module is a simple character device driver that interacts with user space through read and write operations while enforcing certain conditions.
- When the module is inserted using insmod, it takes three parameters: kernel_version_0, kernel_version_1, and time. The driver checks the current kernel version using LINUX_VERSION_CODE and compares it with the values provided by the user. If they do not match, the module is not loaded.
- If the version matches, the driver registers a character device using register_chrdev() and gets a major number dynamically. This information, along with the timer value, is printed in the kernel log and can be viewed using dmesg. At the same time, the starting time is recorded using jiffies.
- After loading, the driver expects two operations from the user. First, the device should be read using a command like cat. This sets a flag indicating that the read operation has been completed. Next, the user writes their username to the device using echo. The driver checks whether the read operation has already happened before allowing the write. The username is copied from user space into the kernel using copy_from_user().
- When the module is removed using rmmod, the driver calculates how much time has passed since it was loaded. It then checks whether the read and write operations were done in the correct order and within the specified time. If all conditions are satisfied, it prints a success message along with the username. Otherwise, it prints a failure message.

- **Key Points:**
- Module insertion fails if kernel version mismatches
- Device must be read before writing
- Username is captured from user space
- Timer ensures execution constraint
- Final validation ensures correctness

---

4. ***Execution Procedure***

Step 1: Compilation :-

- The kernel module is compiled using the Makefile
- This generates: `mymodule.ko` (kernel object file)

```
elias@debian:~/Stuff/CharDevDrivers$ make
make -C /lib/modules/6.12.38+deb13-amd64/build M=/home/elias/Stuff/CharDevDrivers modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.73+deb13-amd64'
  CC [M]  /home/elias/Stuff/CharDevDrivers/mymodule.o
/home/elias/Stuff/CharDevDrivers/mymodule.c: In function 'dev_write':
/home/elias/Stuff/CharDevDrivers/mymodule.c:43:5: warning: ignoring return value of 'copy_from_user' declared with attribute
'warn_unused_result' [-Wunused-result]
   43 |     copy_from_user(username, buf, len);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  MODPOST /home/elias/Stuff/CharDevDrivers/Module.symvers
  CC [M]  /home/elias/Stuff/CharDevDrivers/mymodule.mod.o
  CC [M]  /home/elias/Stuff/CharDevDrivers/.module-common.o
  LD [M]  /home/elias/Stuff/CharDevDrivers/mymodule.ko
  BTF [M] /home/elias/Stuff/CharDevDrivers/mymodule.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.73+deb13-amd64'
```

Step 2: Kernel Version Check :-

- The current kernel version is verified
- This ensures correct parameters are passed during module insertion.

```
elias@debian:~/Stuff/CharDevDrivers$ uname -r
6.12.38+deb13-amd64
```

Step 3: Insert Module :-

- The module is inserted
- Output shows:
    - Successful device registration
    - Major number
    - Timer value

```
elias@debian:~/Stuff/CharDevDrivers$ sudo insmod mymodule.ko kernel_version_0=6 kernel_version_1=12 time=30
elias@debian:~/Stuff/CharDevDrivers$ sudo dmesg | tail
[  101.486925] wlp2s0: authenticated
[  101.491145] wlp2s0: associate with b4:3d:08:5b:dc:d1 (try 1/3)
[  101.502660] wlp2s0: RX AssocResp from b4:3d:08:5b:dc:d1 (capab=0x401 status=0 aid=2)
[  101.502932] wlp2s0: associated
[  156.917784] mymodule: loading out-of-tree module taints kernel.
[  156.917794] mymodule: module verification failed: signature and/or required key missing - tainting kernel
[  156.922836] Given version: 6.12
[  156.922842] Current version: 6.12
[  156.922847] Device registered successfully
[  156.922848] Major=236 Minor=0 Timer=30 sec
```

Step 4: Create Device File :-

- A device file is created in /dev
- This allows user-space programs to interact with the driver

```
elias@debian:~/Stuff/CharDevDrivers$ sudo mknod /dev/mychardev c 236 0
elias@debian:~/Stuff/CharDevDrivers$ sudo chmod 666 /dev/mychardev
elias@debian:~/Stuff/CharDevDrivers$ ls -l /dev/mychardev
crw-rw-rw- 1 root root 236, 0 Apr  1 14:37 /dev/mychardev
```

Step 5: Perform Operations :-

1. **Read:**
   - This triggers the driver's `read()` function

```
elias@debian:~/Stuff/CharDevDrivers$ cat /dev/mychardev
elias@debian:~/Stuff/CharDevDrivers$
```

2. **Write:**
   - This triggers the `write()` function and stores the username

```
elias@debian:~/Stuff/CharDevDrivers$ echo "elias" > /dev/mychardev
elias@debian:~/Stuff/CharDevDrivers$
```

Step 6: Remove Module :-

- This performs final validation and prints result

```
elias@debian:~/Stuff/CharDevDrivers$ sudo rmmod mymodule
elias@debian:~/Stuff/CharDevDrivers$ sudo dmesg | tail
[  436.789849] Device registered successfully
[  436.789850] Major=236 Minor=0 Timer=30 sec
[  441.254956] Device read operation performed
[  446.562446] Username received: elias

[  450.737056] Time elapsed: 13 sec
[  450.737063] Successfully completed the actions within time
[  450.737064] Username: elias

[  450.737069] Device unregistered
```

5. ***Successful Execution***

Output observed:

```
[  450.737063] Successfully completed the actions within time
[  450.737064] Username: elias
```

This confirms:

- Correct order of operations
- Execution within time limit

6. ***Failure Cases***

- *Case 1: Timeout Failure :-*

- Delay introduced beyond time limit

```
elias@debian:~/Stuff/CharDevDrivers$ sudo insmod mymodule.ko kernel_version_0=6 kernel_version_1=12 time=5
elias@debian:~/Stuff/CharDevDrivers$ sleep 10
elias@debian:~/Stuff/CharDevDrivers$ cat /dev/mychardev
elias@debian:~/Stuff/CharDevDrivers$ echo "elias" > /dev/mychardev
elias@debian:~/Stuff/CharDevDrivers$ sudo rmmod mymodule
elias@debian:~/Stuff/CharDevDrivers$ sudo dmesg | tail
[ 3331.629651] Given version: 6.12
[ 3331.629660] Current version: 6.12
[ 3331.629665] Device registered successfully
[ 3331.629666] Major=236 Minor=0 Timer=5 sec
[ 3359.117338] Device read operation performed
[ 3364.898626] Username received: elias

[ 3369.891572] Time elapsed: 38 sec
[ 3369.891583] Failure: Conditions not met
[ 3369.891589] Device unregistered
```

- *Case 2: Wrong Order Failure :-*

- Write attempted before read

```
elias@debian:~/Stuff/CharDevDrivers$ sudo insmod mymodule.ko kernel_version_0=6 kernel_version_1=12 time=30
elias@debian:~/Stuff/CharDevDrivers$ echo "elias" > /dev/mychardev
bash: echo: write error: Invalid argument
elias@debian:~/Stuff/CharDevDrivers$ sudo rmmod mymodule
sudo dmesg | tail
[ 3369.891583] Failure: Conditions not met
[ 3369.891589] Device unregistered
[ 3576.168009] Given version: 6.12
[ 3576.168018] Current version: 6.12
[ 3576.168022] Device registered successfully
[ 3576.168023] Major=236 Minor=0 Timer=30 sec
[ 3581.000442] Write before read not allowed
[ 3588.122399] Time elapsed: 11 sec
[ 3588.122406] Failure: Conditions not met
[ 3588.122410] Device unregistered
```

- *Case 3: Kernel Version Mismatch :-*

```
elias@debian:~/Stuff/CharDevDrivers$ sudo insmod mymodule.ko kernel_version_0=5 kernel_version_1=2 time=30
insmod: ERROR: could not insert module mymodule.ko: Invalid parameters
elias@debian:~/Stuff/CharDevDrivers$ sudo dmesg | tail
[ 3576.168018] Current version: 6.12
[ 3576.168022] Device registered successfully
[ 3576.168023] Major=236 Minor=0 Timer=30 sec
[ 3581.000442] Write before read not allowed
[ 3588.122399] Time elapsed: 11 sec
[ 3588.122406] Failure: Conditions not met
[ 3588.122410] Device unregistered
[ 3829.562200] Given version: 5.2
[ 3829.562209] Current version: 6.12
[ 3829.562211] Kernel version mismatch!
```

Failure :-

- Kernel version mismatch

---

7. ***Observations***

- Kernel modules must match the running kernel version
- Device operations must follow strict order
- Time constraints enforced using kernel timing mechanisms
- `dmesg` is crucial for debugging kernel modules

---

8. ***Conclusion***

A character device driver was successfully implemented with:-

- Parameter validation
- Device interaction
- Time-based execution control
- Proper success and failure handling

The assignment demonstrates practical understanding of Linux kernel module programming and device driver mechanisms.

---

9. ***Files Submitted***

- `mymodule.c`
- `Makefile`
- `README.md`
- `Report.pdf`